winlspci

PowerShell PCI Tool (No Driver)

PCI EXPRESS INVENTORY & LINK DIAGNOSTICS

winlspci User's Manual

`lspci` for Windows — without a kernel driver

A PowerShell tool that enumerates every PCI and PCI Express device in a Windows machine, reports negotiated *and* maximum link speed and width, draws the real bus topology, and answers questions at the attribute level — with no driver to sign, install, or trust.

```
PS> lspci -nn
00:1c.0 PCI bridge [0604]: Intel Corporation Tigerlake PCH-LP PCI
Express Root Port #6 [8086:a0bd] (rev 20)
01:00.0 Non-Volatile memory controller [0108]: SK hynix Gold
P31/BC711/PC711 NVMe SSD [1c5c:174a] (rev 00)
```

Document revision 1.2 · August 2026 · Requires Windows PowerShell 5.1 · MIT Licence
github.com/jpmutschler/winlspci

Contents

A NOTE ON NAMES

winlspci is the name of the tool. The command you type is `lspci`, deliberately, so that muscle memory and existing notes carry over from Linux. Where this manual writes `lspci` in an example, that is the literal command; where it writes winlspci, it means the tool as a whole — the module, its cmdlets and its data files.

NO KERNEL DRIVER · WINDOWS POWERSHELL 5.1 · OFFLINE PCI.IDS · REAL TOPOLOGY · TYPED OBJECTS · JSON / CSV · MIT

1 About winlspci

winlspci answers the question every bench engineer asks first: *what is actually in this machine, and is it running the way it should be?* It does that on Windows, from a normal PowerShell prompt, with nothing installed into the kernel.

1.1 What it is

winlspci is a Windows PowerShell implementation of the familiar Linux `lspci` utility. It lists PCI and PCI Express devices with their vendor, device, subsystem and class identifiers, resolves those identifiers to human-readable names from a bundled copy of the PCI ID database, reports link speed and width — both what the link negotiated and what it is capable of — and can draw the bus as a real parent/child tree.

Beyond that, it does something `lspci` does not: it exposes every field it knows as a typed, queryable attribute, so you can ask precise questions without piping text through a search tool that Windows does not have.

Source and issues: **https://github.com/jpmutschler/winlspci** · MIT · Windows PowerShell 5.1

1.2 Why there is no driver

This is the most important thing to understand about the tool, and the reason its capabilities are shaped the way they are.

> THE CONSTRAINT
>
> **Windows exposes no userland path to PCI configuration space.** A faithful port of `lspci` is therefore impossible without a signed kernel-mode driver — and shipping one of those means driver signing, Secure Boot friction, and a considerably larger attack surface on a lab machine.

Rather than accept that cost, winlspci reads Windows' own PnP/PCI enumeration data instead. That data is populated by the Windows PCI bus driver, which reads configuration space itself at enumeration time. The consequence is a clean trade: everything the bus driver chose to surface is available instantly and safely; everything it did not surface is genuinely out of reach, and winlspci says so rather than guessing.

1.3 Design philosophy

One idea runs through the whole tool, and it is worth stating before the reference chapters:

> THE GOVERNING RULE
>
> **A wrong answer that looks right is worse than an obvious failure.**

That principle is why a value the hardware did not report prints as *not reported* rather than as zero; why a filter that matches nothing returns a non-zero exit code instead of an empty-but-successful result; and why asking for a configuration-space dump produces a clear refusal instead of a partial answer dressed up as a complete one. Chapter 15 covers each of these in detail.

1.4 Who it is for

Hardware validation

Confirm a card enumerated at all, at the right slot, with the right IDs — then confirm the link trained to the speed and width the design calls for.

Field and support

Capture a complete, structured inventory of an unfamiliar machine to CSV or JSON with one command, with no software to install on a customer's system.

Linux engineers on Windows

Keep the flags, the output shape and the reflexes you already have, without pretending Windows has `grep`, `awk` or `cut`.

2 What winlspci can and cannot report

Read this chapter before the reference chapters. The boundary below is not a list of features that have not been written yet — it is the boundary of what any driverless tool on Windows can see.

REPORTED	CANNOT BE REPORTED
Device presence; vendor, device and subsystem IDs; revision	Configuration-space hex dumps (`-x`)
Class, subclass and prog-if, with resolved names	Capability structure **contents** (presence *is* reported)
`[domain:]bus:device.function` — including the PCI segment, where Windows reports one (Hyper-V / Azure)	ASPM state, LTR, DPC
Negotiated *and* maximum link speed and link width	AER register detail (presence only is reported)
Max Payload Size (MPS) and Max Read Request size (MRRS)	Anything requiring a live register read
Driver binding and driver version; device status and problem code	
Device type: root port / upstream or downstream switch port / endpoint / integrated endpoint	
Capability presence: AER, MSI, MSI-X (with vector count), SR-IOV, ACS, ARI, ATS, AtomicOps; and the PCIe capability version	
Physical slot number, firmware location path, device serial number (DSN)	
Power state (most recent D-state) — shown next to a `DOWNTRAINED` flag when it explains it	
Subsystem and programming-interface **names** from `pci.ids` — `Subsystem: Dell PERC H740P [1028:1fd2]`, `(prog-if 02 [NVM Express])`	
NUMA node	
Bus topology (parent/child, via `DEVPKEY_Device_Parent`)	

REPORTED	CANNOT BE REPORTED
Every field above, queryable **per attribute**	

> **THE LINE THAT MATTERS MOST**
>
> Capability *presence* is available; capability *contents* are not. winlspci will tell you a device carries AER, MSI-X with 33 vectors and SR-IOV — and it will not tell you what any of those registers currently hold. Chapter 11 covers what is on the reported side of that line.

2.1 What happens when you ask for the impossible

winlspci does not silently do less than you asked. A request that falls in the right-hand column above produces an explanation and a non-zero exit code:

```
PS> lspci -x
lspci: cannot dump configuration space (-x). Windows exposes no userland path
to PCI config space; that needs a signed kernel-mode driver. ...
```

The command exits with status **2**. A script that assumed `-x` would work therefore fails loudly at the point of the mistaken assumption, rather than quietly producing an inventory with a hole in it. The same exit code covers known lspci flags this tool recognises but has not implemented — see §5.4 and chapter 17.

> **PRACTICAL READING**
>
> If your task is *presence, identity, type, binding, link state, capability presence and topology*, winlspci covers it completely. If your task requires reading a register on the device right now — a capability walk, an ASPM state, an AER error log — no driverless tool can help, and you need a kernel-mode utility from your silicon vendor.

3 Requirements and installation

There is no installer, no service, and no driver. Clone the repository and run it.

3.1 Requirements

Operating system	Windows, with the standard PnP/PCI enumeration available (that is, a normally configured system)
Shell	**Windows PowerShell 5.1** — the version that ships with Windows. No separate runtime to install.
Network	Not required. The PCI ID database is bundled.
Kernel driver	None. That is the point of the tool.
Source and issues	`https://github.com/jpmutschler/winlspci`

> **WHY 5.1 SPECIFICALLY**
>
> winlspci is written to Windows PowerShell 5.1 syntax on purpose — no ternary operator, no `??` null-coalescing, no `-AsHashtable`. The reasoning is simple: **a tool that tells you what is in the machine should not need an install before it runs.** Any Windows box you sit down at can run it as-is.

3.2 Clone and run

```
PS> git clone https://github.com/jpmutschler/winlspci.git $env:LOCALAPPDATA\Programs\winlspci
PS> $env:PATH += ";$env:LOCALAPPDATA\Programs\winlspci\bin"
PS> lspci -nn
```

That gives you the `lspci` command for the current session only. To make it permanent, append to the **User** PATH:

```
PS> $p = [Environment]::GetEnvironmentVariable('PATH','User')
PS> [Environment]::SetEnvironmentVariable('PATH', "$p;$env:LOCALAPPDATA\Programs\winlspci\bin",
'User')
```

> **DO NOT USE SETX**
>
> Use the .NET API shown above, **not** `setx`. The `setx` command silently truncates PATH at 1024 characters, and a developer machine is routinely well past that length. Truncating PATH destroys most of your environment with no warning and no error message.

3.3 Why not `C:\tools`

> **A FOLDER UNDER THE DRIVE ROOT IS WRITABLE BY EVERYONE**
>
> A folder created directly under `C:\` inherits *Modify* for every authenticated user, so any standard account could rewrite `lspci.ps1` for the next administrator who runs it. A hardware-debug tool is exactly the thing that gets run from an elevated shell.
>
> Per-user (`%LOCALAPPDATA%\Programs`) or `C:\Program Files` are both fine. The commands above use the former.

3.4 From the PowerShell Gallery

Once published (see `packaging/README.md`), the module installs into a `PSModulePath` directory — which has no `bin\` folder on PATH, so the `lspci` command needs putting there separately:

```
PS> Install-Module winlspci -Scope CurrentUser
PS> Install-LspciShim          # -Remove to undo
PS> lspci -nn
```

`Install-LspciShim` writes a two-line `lspci.cmd` into `%LOCALAPPDATA%\Microsoft\WindowsApps`, pointing at the installed CLI. That directory is usually on the user's PATH — the command warns if it is not — and is

writable only by that user, which is the same reasoning as §3.3.

3.5 Importing the module directly

If you would rather work with the cmdlets than the `lspci` command wrapper, import the module and call them:

```
PS> Import-Module $env:LOCALAPPDATA\Programs\winlspci\winlspci.psd1
PS> Get-PciDevice -Device '10de:' | Format-Lspci -Verbosity 2
```

The cmdlets return real PowerShell objects, so everything downstream of them — `Where-Object`, `Select-Object`, `Export-Csv`, comparison operators on numeric fields — behaves the way you expect. Chapter 7 builds on this.

3.6 Verifying the installation

```
PS> lspci -nn          # should print one line per device
PS> lspci -Version     # tool version, and the date of the bundled pci.ids
PS> lspci -ListAttributes   # should print the queryable attribute names
PS> (lspci).Count      # device count — a sanity check against Device Manager
```

4 Quick start

Six commands that cover most of what the tool is used for.

4.1 List everything, with names and IDs

```
PS> lspci -nn
00:1c.0 PCI bridge [0604]: Intel Corporation Tigerlake PCH-LP PCI Express Root Port #6
[8086:a0bd] (rev 20)
01:00.0 Non-Volatile memory controller [0108]: SK hynix Gold P31/BC711/PC711 NVMe SSD
[1c5c:174a] (rev 00)
```

4.2 Look closely at one vendor's devices

```
PS> lspci -d 1c5c: -vv
01:00.0 Non-Volatile memory controller: SK hynix Gold P31/BC711/PC711 NVMe Solid State Drive
(rev 00)
        LnkSta: 8GT/s x4 (max 8GT/s x4)
        Type: PCIe Endpoint (PCIe capability v2)
        Driver: stornvme (10.0.26100.8972)
        Subsystem: SK hynix [1c5c:174a]
        DevCtl: MPS 256 bytes (max 512), MaxReadReq 512 bytes
        Capabilities: AER, MSI, MSI-X (33 vectors)
        Device Serial Number 00-00-00-00-00-00-00-00 (capability present, not populated)
        Power: D0 (most recent state Windows recorded)
        Location: PCIROOT(0)#PCI(0600)#PCI(0000)
        InstanceId: PCI\VEN_1C5C&DEV_174A&SUBSYS_174A1C5C&REV_00\4&391CC7C&0&0030
```

Note the `LnkSta` line: the negotiated speed and width, and in parentheses the maximum the link is capable of. Those two numbers agreeing is the answer to most link questions; them disagreeing is the beginning of an investigation. Chapter 11 covers the `Type`, `Capabilities`, serial-number, power and location lines.

4.3 Investigate one device that is running slow

```
PS> lspci -s f3:00.0 -v
f3:00.0 3D controller: NVIDIA Corporation GA107M [GeForce RTX 3050 Ti Mobile] (rev a1)
        LnkSta: 8GT/s x4 (max 16GT/s x16)  ⟵ DOWNTRAINED (speed, width)
                (device is in D3: may be idle link power management rather than a fault)
        Type: PCIe Endpoint (PCIe capability v2)
        Driver: nvlddmkm (32.0.16.1088)
```

This is the tool at its most useful: it names the problem, names the most likely innocent explanation for it, and leaves the conclusion to you.

4.4 See the tree

```
PS> lspci -t
-[0000:00]-+-00:00.0  Tiger Lake-UP3/H35 4 cores Host Bridge/DRAM Registers
           +-00:06.0-[01]  11th Gen Core Processor PCIe Controller  [root port]
           |           \-01:00.0  Gold P31/BC711/PC711 NVMe Solid State Drive  (8GT/s x4)
           \-00:1d.0-[f3]  500 Series Chipset Family PCI Express Root Port #9  [root port]
                       \-f3:00.0  GA107M [GeForce RTX 3050 Ti Mobile]  (8GT/s x4)
```

One root per `[domain:bus]`, bridges carrying the bus range beneath them and a port-type tag, endpoints with their link state inline. Chapter 9 reads this in full.

4.5 Find fast links — without grep

```
PS> lspci -Attribute LinkSpeed -Match '16GT|32GT|64GT'   # no grep needed
```

4.6 Find anything running below its maximum

```
PS> lspci -Downtrained                                  # anything below its max
```

> **INTERPRETING -DOWNTRAINED**
>
> A downtrained link is a *finding*, not automatically a fault. On a laptop, a GPU reporting 8GT/s x4 against a maximum of 16GT/s x16 is almost certainly link power management at idle — which is why the power state is printed alongside it when the device is in D1–D3. winlspci tells you what the link is doing; deciding whether that is a problem is still yours.

5 Command reference — standard lspci flags

Measured against Linux `lspci`, not against an aspiration. Three lists follow: what is implemented, what is refused on principle, and what is recognised but not built.

5.1 Implemented

LSPCI	WINLSPCI	NOTES
`-s [[<dom>]:]<bus>:]<dev>[.<fn>]`	`-s`	lspci's grammar, with every field optional: `01:` (bus), `01:00.0`, `1` (**device** 01 on any bus), `.0` (every function 0), `:00.0`, `0000:01:00.0`. Padded or unpadded. A selector *without* a domain matches every domain; `556f:00:02.0` matches only that one
`-d [<ven>]:[<dev>][:<class>]`	`-d`	Vendor and device are **exact hex IDs** — `-d 80:` is vendor `0080`, not every `80xx`. Class is a *prefix*: `::01` is all storage. Examples: `-d 11f8:`, `-d :174a`, `-d ::0108`
`-t`	`-t`	Real topology, built from `DEVPKEY_Device_Parent`. Shows link state inline; bridges tagged `[root port]` / `[upstream port]` / `[downstream port]`
`-v`	`-v`	Physical slot; link state (current **and** maximum, with the power state when it explains a downtrain); device type; driver; status
`-vv`	`-vv`	Adds MPS, MRRS, subsystem (as `vendor:device`), NUMA node, capability presence (`Capabilities: AER, MSI-X (33 vectors), SR-IOV …`), `ACS: present | not needed | missing`, serial number, power state and location path — see chapter 11
`-vvv`	`-vvv`	Everything `-vv` shows, then every property Windows exposes, **plus an explicit list of what is missing**
`-n`	`-n`	Numeric IDs only
`-nn`	`-nn`	Names and IDs together

LSPCI	WINLSPCI	NOTES
`-D`	`-D`	Domain on every line. `0000` on ordinary machines. Hyper-V and Azure SR-IOV functions carry the PCI segment in the upper bits of Windows' bus number (`PCI bus 5598976` = segment `556f` , bus `00`); those always show it, as lspci does: `556f:00:02.0` . See chapter 10
`-k`	`-k`	Driver and version — the same lines `-v` shows
`-m / -mm`	`-m / -mm`	lspci's machine-readable forms: one quoted line per device, or `Key:\tValue` records. Quoted and escaped the way lspci does it — unlike `-Delimited` , these *are* meant to be parsed by other tools
`-i <file>`	`-i <file>`	Use an alternate `pci.ids`
`-tv, -nnk, -vvnn …`	same	Short flags combine, as in lspci. They are **case-sensitive**: `-D` ≠ `-d`

SHORT FLAGS VERSUS LONG OPTIONS

Short lspci flags are case-sensitive, because `-D` and `-d` are two different things in lspci and must stay that way here. The long options that are unique to this tool — `-Json` , `-Downtrained` , `-Attribute` and the rest — are case-*in*sensitive and may be abbreviated to any unique prefix.

5.2 Values and the PowerShell parser

ONLY RELEVANT IF YOU BYPASS THE LAUNCHER

Run through the `lspci` launcher (`bin\lspci.cmd` , the one on PATH) and every form works exactly as typed, because the script recovers the raw command line.

If you call `bin\lspci.ps1` directly from a PowerShell *prompt*, PowerShell's own parser gets there first: it splits `-s01:` at the colon, and reads a bare `.0` or `00.0` as the number `0` . At the prompt, put a space before the value and quote anything beginning with a dot or a colon.

```
PS> .\lspci.ps1 -s '.0'
PS> .\lspci.ps1 -s ':00.0'
PS> .\lspci.ps1 -s 01:00.0
```

5.3 Deliberately refused

LSPCI	WHY
`-x, -xxx, -xxxx`	Configuration-space dumps. **Impossible without a signed kernel-mode driver.** winlspci exits 2 with an explanation rather than pretending otherwise.

5.4 Not implemented (possible, just not built)

These are absent by omission rather than by principle. Nothing about the driverless approach prevents them.

FLAG	PURPOSE
`-b`	Bus-centric view — Windows exposes no bus-relative view distinct from the CPU one
`-p`	Custom ID file — `-i` loads an alternate `pci.ids`
`-P`	Path display
`-q, -Q`	Online ID lookup
`-A, -O, -F, -G, -H1, -H2`	Access-method selection — meaningless in a driverless implementation
`-M`	Bus mapping

Each of these is **recognised**. Passing one exits 2 with *"not implemented"* and a pointer to the nearest equivalent. Anything else unknown exits **64**.

> **WHY RECOGNISE A FLAG YOU DO NOT SUPPORT**
>
> An earlier version let PowerShell's parameter binder guess, and the results were quietly wrong: `-p` ran `-PresentOnly`, and `-m` demanded a value for `-Match`. **A tool that runs a different command from the one you typed is worse than one that says no.**

6 Command reference — flags beyond lspci

These flags have no Linux counterpart. They exist because PowerShell's object pipeline makes possible what a text pipeline cannot do safely. All of them are case-insensitive and may be abbreviated to a unique prefix.

FLAG	WHY IT EXISTS
`-Attribute <names>`	Query at the attribute level rather than the device level. Wildcards are accepted: `-Attribute Link*`
`-Match <regex>`	Filter by **value** — the job you would otherwise have handed to `grep`
`-PresentOnly`	Drop attributes the device does not report, leaving only real data
`-ListAttributes`	Discover what is queryable on this build. Answered from the module's static list, so it does not enumerate the machine
`-Delimited`	`|`-separated records, for `-split` / `cut` / `awk` habits
`-Delimiter`	Change the separator — for example a tab, giving TSV
`-Header`	Name the columns in delimited output
`-Json`	Structured JSON output. The envelope carries `schemaVersion` (currently 1), `winlspciVersion`, `generatedAt` and `computerName`, so two machines can be diffed — see the note below

FLAG	WHY IT EXISTS
`-Csv`	Structured CSV output, correctly quoted
`-Downtrained`	Only devices below their maximum speed or width — the `Downtrained` property. This is a *filter*: it exits 1 when nothing is downtrained
`-Baseline <file>` `-Diff <file>`	Save the enumeration; later, report what appeared, disappeared or changed — exit 3 if anything did. See chapter 12
`-IncludeVolatile` `-IgnoreAttribute`	Compare `PowerState` too; or leave out attributes you expect to move. Chapter 12
`-Watch <seconds>` `-Iterations <n>`	Re-enumerate on an interval and print only the changes, timestamped. Chapter 12
`-ComputerName <a,b,c>` `-Credential`	Enumerate other machines over WinRM. See chapter 13
`-Version`	Tool version, and the date of the bundled `pci.ids`

THE JSON ENVELOPE NAMES YOUR HOST

`computerName` is in the envelope so that two captures can be told apart and diffed. It also means a `-Json` capture identifies the machine it came from — worth remembering before pasting one into a public issue.

Schema versioning: additive changes keep the same `schemaVersion`; a rename, or a change in what a field *means*, bumps it. A consumer can therefore trust version 1 to keep meaning what it meant.

CHOOSING BETWEEN THEM

Reach for `-Attribute` and `-Match` when you want a specific answer, `-Csv` or `-Json` when something else will consume the output, and `-Delimited` when you already know the one-liner you want to write.

7 Attribute-level queries

Windows has no `grep`, so `lspci -vv | grep LnkSta` has no equivalent. Rather than bolt a text search onto the tool, winlspci serialises the data to one record per attribute — which turns out to be strictly better than the thing it replaces.

7.1 One record per attribute

```
PS> lspci -Attribute Link* -s 01:00.0 -PresentOnly

Slot    VendorId DeviceId Attribute         Value Present
----    -------- -------- ---------         ----- -------
01:00.0 1c5c     174a     LinkStateReported  True    True
01:00.0 1c5c     174a     LinkSpeed         8GT/s    True
01:00.0 1c5c     174a     LinkWidth             4    True
```

7.2 Filtering by value

This is the part you would have piped to `grep`:

```
PS> lspci -Attribute LinkSpeed -Match '16GT|32GT|64GT'   # every Gen4+ link
PS> lspci -Attribute Driver -Match 'stornvme'            # everything on one driver
PS> lspci -Attribute '*' -Match '11f8' -Csv              # every field mentioning the vendor
PS> lspci -ListAttributes                                # what can I ask for?
```

7.3 Three properties that beat text search

`Present` is a field

Absent and zero never collapse into each other. A root port that reports no link width is not a device running at x0 — and a text pipeline cannot tell those apart.

Values keep their types

`LinkWidth` is an integer, so `-lt 4` is a numeric comparison. `grep` would have you comparing strings.

No match exits 1

An attribute query that matches nothing returns a non-zero exit code, so a script can tell "no match" from "it worked".

7.4 Composing with the pipeline

Because these are real objects, ordinary PowerShell applies:

```
# every device not running at full width, as a table
Get-PciDevice | Where-Object { $_.LinkStateReported -and $_.LinkWidth -lt $_.MaxLinkWidth } |
    Select-Object Slot, DeviceName, LinkWidth, MaxLinkWidth

# diff two machines
Get-PciDevice | ConvertTo-PciAttributeRecord | Export-Csv machine-a.csv
```

Exporting the attribute records from two systems and diffing the two CSV files is the fastest way to answer "what is different about this machine?" — a question that is otherwise a long afternoon.

8 Delimited output and coming from Linux

You cannot have `grep`, `awk` and `cut` here, but you can have the shape of them.

8.1 Delimited records

`-Delimited` emits one record per line with `|` between fields:

```
PS> lspci -Delimited -s 01:
01:00.0|1c5c|174a|0108|Non-Volatile memory controller|SK hynix|Gold P31 NVMe
SSD|00|8GT/s|4|8GT/s|4|stornvme|OK
```

`-Header` names the columns; `-Delimiter "`t"` gives you TSV.

8.2 Translation table

LINUX	WINLSPCI
`lspci | grep NVMe`	`lspci | Select-String NVMe`
`lspci -vv | grep LnkSta`	`lspci -Attribute LinkSpeed,LinkWidth`
`lspci | cut -d' ' -f1`	`lspci -Delimited | %{ ($_ -split '|')[0] }`
`lspci | awk -F'|' '{print $1, $9}'`	`lspci -Delimited | %{ $f=$_ -split '|'; "$($f[0]) $($f[8])" }`
`lspci | wc -l`	`(lspci).Count`
`lspci -d 11f8: || echo "absent"`	Same — a filter matching nothing exits 1

8.3 Two properties worth relying on

An empty field means “not reported”; a literal `0` means zero

The distinction the object model protects survives into the text form for free, because an unset value serialises to nothing at all. This is real output from the machine winlspci was written on:

```
00:1e.0|8086|a0a8|0780|Communication controller|Intel Corporation|Tiger Lake-LP Serial IO UART Controller #0|20|||||iaLPSS2_UART2_TGL|OK
    the four consecutive empty fields above are the link-state columns: not reported
f3:00.0|10de|25a0|0302|3D controller|NVIDIA Corporation|GA107M [GeForce RTX 3050 Ti Mobile]|a1|8GT/s|4|16GT/s|16|nvlddmkm|OK
```

A device with no link state and a device genuinely training at x0 are different findings, and `grep`-shaped output normally destroys that difference. Here it survives a `-split`.

> **THE SECOND LINE, READ CLOSELY**
>
> That GPU reports **8GT/s x4 against a maximum of 16GT/s x16** — which is exactly why `-Downtrained` exists. On a laptop it is almost certainly link power management at idle rather than a fault, and that is the point: the tool tells you what the link is doing, and the judgement stays with you.

The delimiter is stripped from values, not escaped

Quoting rules turn a one-liner into a parser, which defeats the purpose — so a `|` appearing inside a value becomes `/` and the column count stays fixed. No `|` occurs in live PCI data or in `pci.ids` today, but `FriendlyName` comes from driver INF files and is not under the tool's control.

> **IF YOU NEED REAL QUOTING**
>
> Use `-Csv`. It quotes properly. `-Delimited` is a convenience for known-shaped pipelines, not a general-purpose serialisation format.

> **STRUCTURED BEATS TEXTUAL WHERE IT CAN**
>
> `lspci -Attribute LinkSpeed -Match '16GT|32GT|64GT'` keeps types, so `LinkWidth -lt 4` is a numeric comparison rather than a string one. Reach for `-Delimited` when you already know the pipeline you want to write.

9 Topology

`-t` builds a real tree from `DEVPKEY_Device_Parent`, in lspci's shape: one root per `[domain:bus]`, bridges annotated with the bus range their descendants occupy, children under the bridge token, and link state inline — which is where a downtrained device becomes obvious.

```
-[0000:00]-+-00:00.0  Tiger Lake-UP3/H35 4 cores Host Bridge/DRAM Registers
           +-00:06.0-[01]  11th Gen Core Processor PCIe Controller  [root port]
           |           \-01:00.0  Gold P31/BC711/PC711 NVMe Solid State Drive  (8GT/s x4)
           +-00:1c.0-[f2]  500 Series Chipset Family PCI Express Root Port #6  [root port]
           |           \-f2:00.0  Wi-Fi 6 AX200  (5GT/s x1)
           \-00:1d.0-[f3]  500 Series Chipset Family PCI Express Root Port #9  [root port]
                       \-f3:00.0  GA107M [GeForce RTX 3050 Ti Mobile]  (8GT/s x4)
```

9.1 Reading the tree

`-[0000:00]-`	A root: one per `[domain:bus]`. On an ordinary machine there is one; see chapter 10 for what puts more than one here
`00:06.0-[01]`	A bridge, with the secondary/subordinate bus range its descendants occupy in brackets
`[root port]`	The device type tag — also `[upstream port]` and `[downstream port]`. See chapter 11
`(8GT/s x4)`	The endpoint's current link state, as *speed × width*

A switch renders as its full hierarchy — `01:00.0-[02-04] … [upstream port]`, then `02:01.0-[03] … [downstream port]`, and the endpoints under each. That is what the type tags and bus ranges are for: on a fabric with switches, the shape of the tree *is* the answer you came for.

9.2 Where it deliberately differs from lspci

- Every device keeps its full `bus:device.function` and stays on a line of its own. lspci collapses a single-child bridge onto one line; this does not.
- Descriptions are shown, not just addresses.
- A device whose parent is not itself a PCI device becomes a **root** rather than being dropped, and a node the walk cannot reach — a parent cycle — is rendered as a root of its own.

WHY A RAGGED TREE IS THE RIGHT FAILURE

An incomplete tree that *looks* complete is worse than an obviously ragged one. “Every enumerated device appears exactly once” is pinned by a test, so what you see is the whole inventory even when the hierarchy is untidy.

9.3 Whose link is it? Downstream attribution

Windows reports link state on the *downstream device*, not on the bridge. A bridge with exactly one child that reports a link therefore shows that child's link, in three explicitly named fields — `DownstreamSlot`, `DownstreamLinkSpeed`, `DownstreamLinkWidth` — and `-v` spells out where the number came from:

```
        LnkSta: not reported by this device (downstream 01:00.0 reports 8GT/s x4)
```

Whose link it is stays explicit. A root port silently borrowing its child's numbers would be a convenient lie of exactly the kind §15.1 exists to prevent.

A CONSEQUENCE WORTH KNOWING: -S AND -D ARE *DISPLAY* FILTERS

Because downstream attribution needs the whole picture, the entire machine is enumerated and the selector is applied **last**. So `lspci -s 00:1c.0 -v` shows the same downstream link as the full listing would — and a filtered query costs about the same as an unfiltered one (chapter 16). Filtering here buys you a shorter answer, not a faster one.

10 PCI domains (segments)

On an ordinary machine every device is in domain `0000`, the slot is `bus:device.function`, and `-D` is the only way to see the domain — exactly as in lspci. Virtualised hardware is the exception, and it is worth understanding before you meet it.

10.1 Where the segment hides on Hyper-V and Azure

Hyper-V and Azure report SR-IOV functions on a “bus” that is really segment and bus packed together. Windows says `PCI bus 5598976`, which is `0x556F00` — and since a PCI bus number is only 8 bits wide, the upper bits are the segment.

HOW THIS WAS FOUND

An earlier version printed that value as bus `556f00`: a slot nothing could parse. The first CI run on a GitHub Windows runner — an Azure VM with one Mellanox ConnectX-4 virtual function — found it. Hardware nobody owns is exactly the hardware a test machine will hand you.

The bus number is now split into a `Domain` property (the upper 16 bits) and the bus itself (the low 8). As lspci does, a non-zero domain is always shown in the slot:

```
PS> lspci -nn                       # on an Azure VM
3851:00:00.0 Non-Volatile memory controller [0108]: Microsoft Corporation Standard NVM Express
Controller [1414:b111] (rev 01)
7870:00:00.0 Ethernet controller [0200]: Microsoft Corporation Microsoft Azure Network Adapter
Virtual Bus [1414:00ba] (rev 00)
c05b:00:00.0 Non-Volatile memory controller [0108]: Microsoft Corporation ASAP NVM Express
Controller [1414:00a9] (rev 00)
```

10.2 The rules, in full

Slot format	`[dddd:]bb:dd.f` — the domain appears only when there is one
The `Domain` property	An integer (`0` on an ordinary machine), and one of the queryable attributes: `lspci -Attribute Domain`
Selecting without a domain	`-s` without a domain matches *every* domain — `-s 00:00.0` finds all three devices above
Selecting one domain	`-s 7870:00:00.0` finds exactly one. A domain nobody is in matches nothing and exits 1
What `-D` does	Prefixes `0000:` only where the slot has no domain already. It never produces `0000:7870:…`
Sort order	By the full slot string, so devices group by domain
Where the numbers come from	Which domains exist is Windows' decision. The numbers are the same ones Linux `lspci` shows on the same VM

> **IF YOU ONLY EVER TOUCH PHYSICAL MACHINES**
>
> You will see `0000` and nothing else, and none of this changes how you use the tool. It matters the first time you run winlspci inside a VM with passed-through or SR-IOV devices — where a slot that suddenly has four extra hex digits in front of it is correct, not corrupt.

11 Device type, capabilities, slot and power

Four groups of fields that go well beyond presence and link state. All of them come from DEVPKEYs that the PCI bus driver fills from configuration space at enumeration — so they cost nothing extra in trust, and about 300 ms extra per full listing, which chapter 16 accounts for.

11.1 Device type

`DeviceType` is what Windows says the device *is*:

PCIE ROOT PORT · PCIE UPSTREAM SWITCH PORT · PCIE DOWNSTREAM SWITCH PORT · PCIE ENDPOINT · PCIE ROOT COMPLEX INTEGRATED ENDPOINT · CONVENTIONAL PCI

`IsBridge` is derived from it. `-t` tags the bridges with their type, and a device that reports no link state now says *why* when the type explains it — an integrated endpoint or a conventional PCI device has no PCIe link to report — rather than falling back on the generic "not reported".

11.2 Capability presence

`-vv` prints one line naming the capabilities the device carries:

```
Capabilities: AER, MSI, MSI-X (33 vectors), SR-IOV, ARI, ATS, AtomicOps
```

> **PRESENCE, NOT CONTENTS**
> *Which* capabilities a device carries is in the PnP data. What their registers *say* is not, and `-vvv` continues to say so. This is the same boundary as chapter 2: a capability walk needs a live register read.

Each flag is also a boolean attribute — `-Attribute *Capable, Msi*` — and is `$null` when Windows reported nothing at all, which is not the same as reporting "absent".

SR-IOV is reported only for devices that actually carry the capability, and its value is a status rather than a boolean: `SriovStatus` is `ok`, or a reason virtual functions cannot be enabled.

11.3 ACS — and why it gets its own line

Access Control Services appears on its own `-vv` line with one of three values:

`ACS: present`	The port carries a populated ACS capability register
`ACS: not needed`	Root-complex integrated endpoints, which have no peer-to-peer path to isolate
`ACS: missing`	On a bridge: the port has no ACS capability

It is a statement about the *port*, and "missing" does not belong in a list of things a device *has* — which is why it is not folded into the `Capabilities:` line. An endpoint without ACS is the normal case and says nothing about isolation, since the ports above it decide that, so the line is not printed for endpoints at all; the `AcsSupport` attribute still carries the value.

> **IF YOU ARE REASONING ABOUT IOMMU GROUPS OR DMA ISOLATION**
> This is the line. The three values were verified against the ACS capability register on real hardware rather than read off an enum: every `present` port carries a populated register, and every `missing` one has none.

11.4 Physical slot, location path and serial number

`PhysicalSlot`	The chassis slot number, from `DEVPKEY_Device_UINumber`. Printed as lspci's `Physical Slot:`
`LocationPath`	The firmware location path, e.g. `PCIROOT(0)#PCI(1C00)#PCI(0000)`
`SerialNumber`	The Device Serial Number (DSN) capability, formatted `00-11-22-33-44-55-66-77`

11.5 Power state

`PowerState` is the most recent D-state Windows recorded for the device, from `DEVPKEY_Device_PowerData`. It shows in `-vv`, and it is appended to a `DOWNTRAINED` flag when the device is in D1–D3:

```
        LnkSta: 8GT/s x4 (max 16GT/s x16)  ⟵ DOWNTRAINED (speed, width)
                (device is in D3: may be idle link power management rather than a fault)
```

> **WHY IT IS PHRASED AS A SUGGESTION**
>
> It is a snapshot, not a live read — the device may have woken since Windows recorded it. That is why it is offered as the *likely explanation* and never as a verdict. The judgement stays yours, which is the same position chapter 4 takes on `-Downtrained` generally.

12 Baseline, diff and watch

The attribute-record shape was always meant for “did anything change?” Chapter 7 gets you as far as exporting two CSVs and comparing them by hand; this chapter is the tool finishing the job.

12.1 The basic loop

```
PS> lspci -Baseline before.json          # 24 devices written
   ... reboot / flash firmware / swap a card ...
PS> lspci -Diff before.json
~ 01:00.0  LinkWidth: 4 → 1
- 02:00.0  gone: 8086:15f3 Ethernet Controller I225-V
+ 03:00.0  appeared: 10de:2684 AD102 [GeForce RTX 4090]
PS> $LASTEXITCODE                        # 3: differences found (0 = identical)
```

Three markers, and they mean what they look like: `~` changed, `-` disappeared, `+` appeared.

12.2 How devices are matched

Devices are matched by `InstanceId`. A reseated card whose instance id changed but whose slot and IDs did not is reported as `Changed(InstanceId)` — *not* as a removed device plus an added one, which is the same card and would read as two findings instead of one.

> **ABSENT STAYS DISTINCT FROM ZERO HERE TOO**
>
> An attribute that went from *not reported* to `0` is reported as `<absent> → 0`. The rule from §15.1 survives all the way into the diff output, where it matters most — that transition is a real change, and collapsing it would hide exactly the kind of thing you ran a diff to find.

A newer baseline's extra attributes are reported too, as `value → <absent>`; the comparison walks the union of both sides rather than only the keys the older file happens to have.

12.3 What is ignored, and what you can ignore

Ignored by default	Only `PowerState`. It flips with idle, and would answer "did the update change anything?" with D-state noise every single time. Everything else is compared, because the point is to see what moved
`-IncludeVolatile`	Compare `PowerState` as well
`-IgnoreAttribute`	Names, with wildcards: `DriverVersion` after an intended update, `Link*` if you only care about presence

```
PS> lspci -Diff before.json -IgnoreAttribute DriverVersion,Link*
```

12.4 Selectors and output formats

- A `-d` or `-s` selector applies to **both sides** — `lspci -d 1c5c: -Diff before.json` diffs just the NVMe against the baseline's NVMe. A selector matching nothing is still exit 1.
- `-Diff` takes `-Json`, `-Csv` and `-Delimited` for scripts.
- A baseline file is the same envelope `lspci -Json` writes, so **either serves as the other** — you can diff against a JSON capture somebody sent you.

12.5 Watching for changes live

```
PS> lspci -Watch 2                # re-enumerate every 2 s, print only changes
PS> lspci -Watch 2 -Iterations 30 # stop after 30 passes
```

`-Watch` re-enumerates on an interval and prints only what changed, timestamped, until Ctrl-C — which is what you want for hot-plug, retimer bring-up and link flaps. `-Iterations n` stops after *n* passes, and exit 3 still means something changed.

> **TWO SECONDS IS THE FLOOR**
>
> Each pass is a full enumeration (chapter 16), so an interval much below ~2 s asks for work that cannot finish before the next pass begins.

12.6 From the module

`Export-PciBaseline`	Write a baseline file
`Compare-PciBaseline`	Compare a file against the machine now
`Compare-PciDeviceSet`	Compare any two sets of device objects

13 Remote machines

Everything the module reads comes through `Get-CimInstance` and `Invoke-CimMethod`, and both take a CIM session — so a fleet is one parameter away.

```
PS> lspci -ComputerName node1,node2,node3 -Downtrained
node2: 81:00.0 Non-Volatile memory controller: Samsung ... (rev 00)
        LnkSta: 8GT/s x2 (max 16GT/s x4)  ⟵ DOWNTRAINED (speed, width)

PS> Get-PciDevice -ComputerName node1,node2,node3 | Where-Object Downtrained |
        Select-Object ComputerName, Slot, LinkSpeed, MaxLinkSpeed, LinkWidth, MaxLinkWidth
```

13.1 Sessions and credentials

One WinRM session is opened per name. `-Credential <user>` *prompts* rather than taking a password on the command line, so it needs a console; from a script, call `Get-PciDevice -Credential $cred` with a `PSCredential` you already hold.

`Get-PciDevice -CimSession $s` takes sessions you manage yourself. DCOM works too — `New-CimSessionOption -Protocol Dcom` — which is also how the test suite exercises the remote path on a machine without WinRM.

13.2 What happens when a node misbehaves

A node cannot be reached	Reported with a warning and skipped
No node can be reached	The command **fails with exit 70** rather than printing an empty list that looks like “no devices”
A blank name	Refused, rather than quietly meaning “this machine”
A duplicate name	Enumerated once
Property fetch fails for every device on a node	Warned about, and the node is dropped

> **THE SAME INSTINCT AS EVERYWHERE ELSE**
>
> An empty result that looks like success is the failure this tool works hardest to avoid (§15.2). Across a fleet the stakes are higher, not lower: “none of the nodes answered” and “none of the nodes has that card” must never render identically.

13.3 How multi-host output is shaped

Every object carries `ComputerName` — locally it is this machine's name — so one pipeline can sort, group and diff across hosts. With more than one host the CLI also:

- prefixes each device line with `host:`;
- prints one `-t` tree per host;
- leads `-Delimited` output with a `ComputerName` column.

13.4 Two things worth knowing

> **WHERE NAMES ARE RESOLVED, AND HOW FAST IT RUNS**
>
> `pci.ids` names are resolved on the machine *running* the tool, not on each target — so a fleet gets consistent naming from your one bundled database, and no node needs the tool installed.
>
> The remote property fetch is **serial**: CIM sessions are not shared across runspaces, so the parallel path described in §16.4 does not apply to remote targets.

14 The PCI ID database

Vendor and device names come from `pci.ids`, maintained by the PCI ID Project. winlspci bundles a copy so that it works with no network — which, on a lab bench, is usually the situation.

Bundled location	`data/pci.ids`
Approximate size	~1.4 MB
Source	The PCI ID Project — `https://pci-ids.ucw.cz`
Licence	BSD / GPL dual-licensed
Network needed to run	No
Date of bundled copy	Shown by `lspci -Version`

14.1 Refreshing the database

Updates are explicit — the tool never fetches in the background:

```
PS> Update-PciIds          # from https://pci-ids.ucw.cz
```

The update is written so that a bad download cannot leave you worse off than before it ran:

- It downloads to the **temp directory**, not over the live database.
- It **proves the file parses** as a PCI ID database — vendors are present, a known vendor resolves, a known class resolves — before touching anything.
- It keeps the previous copy as `pci.ids.bak`.
- It forces **TLS 1.2** on for older Windows PowerShell hosts, which otherwise negotiate a protocol the server no longer accepts.

A truncated fetch over a captive-portal network would otherwise leave you with a tool that reports numeric IDs and no names, at the exact moment you need names.

14.2 When a name looks wrong

> **VENDOR ID 11F8 IS MICROCHIP, WHATEVER NAME THE DATABASE SHOWS**
>
> Microsemi acquired PMC-Sierra; Microchip acquired Microsemi; the vendor ID never changed. Databases from before mid-2026 still read **"PMC-Sierra Inc."**, so a Microchip Switchtec switch showed as PMC-Sierra. The bundled copy now says **"Microchip Technology"**.
>
> A test accepts any of the three names, so the answer stays stable whichever vintage of `pci.ids` you are running — and nobody at a bench at 11pm concludes they have the wrong part.

The general lesson holds beyond this one case: a vendor name in `pci.ids` records who registered the ID, not who sells the silicon today. When a name surprises you, check the acquisition history before you check the hardware.

15 Design principles

Three decisions shape how winlspci behaves in the cases that matter. All three come from the same instinct: **a wrong answer that looks right is worse than an obvious failure.**

15.1 "Not reported" is never rendered as zero

Root ports and chipset devices legitimately expose no link state. Rendering that as `x0` would look like a dead link — a false alarm that costs somebody an hour. So an absent value stays `$null` internally and prints as *"not reported by this device"*. `Present` is a real field in attribute output for the same reason.

The same rule covers two related cases:

Missing class codes	The host bridge reports no class property at all, so the class is read from the `CC_0600` hardware ID instead. A device with no class from either source prints *Unknown class* `[????]` rather than class `0000`, which would be a real class code the device does not have.
The downtrained flag	`DOWNTRAINED` is only ever said when *both* the current and the maximum are reported. In PowerShell `$null -lt 4` evaluates to true, and an earlier version used that comparison directly — so it flagged a width that was never there.

There are tests for each of these.

15.2 A filter that matches nothing exits non-zero

`lspci -d 11f8:` finding no card must not look like success. That is the difference between *"the card is not there"* and *"the command worked"*, and a build script needs to be able to tell them apart. The same rule applies to `-Downtrained` and to an attribute query that returns no rows.

15.3 Selectors are parsed, not string-matched

`-s` follows lspci's grammar field by field rather than comparing against the raw address string. The consequences are all things that used to be wrong:

- `1:` and `01:` agree, because both are parsed as bus 01 rather than compared as text.
- `-s 1` means **device 01 on any bus**, as it does in lspci. An earlier version read it as bus 01 — the same string, a different question.
- `.0` and `:00.0` work, selecting by function and by device.function.

`-d` compares vendor and device as **hex numbers**, so `-d 8:` no longer quietly returns every Intel device. Anything that is not valid hex produces one clear error, rather than a .NET exception per device.

Absent ≠ zero
Missing data prints as missing, never as a value that looks like a fault.

Empty ≠ success
No match is a non-zero exit, so automation can branch on it correctly.

Selectors are parsed
A slot or ID selector means what it means in lspci, not what string comparison happens to do.

16 Performance

Full enumeration of a laptop (24 devices) takes about 1.7 seconds in-process, or about 2.5 seconds as a fresh `lspci` command — of which roughly 0.4 seconds is PowerShell starting up and importing the module.

~1.7 s
Full enumeration, in-process

~2.5 s
Fresh `lspci` command

~0.6 s
`-ListAttributes`

~30 ms
Per-device property fetch

`-ListAttributes` is quick because it answers from the module's static list rather than enumerating the machine at all.

16.1 How it got there

Reaching those numbers took several attempts. Two of them are worth recording.

APPROACH	TIME
`Get-PnpDevice` + per-property `Get-PnpDeviceProperty`	**> 5 min**
Batched `Get-PnpDeviceProperty -KeyName <16 keys>`	~36 s
`Get-PnpDeviceProperty -KeyName '*'`	*appeared* 2× faster — **returned zero properties**
`Win32_PnPEntity ( SELECT * )` + `Invoke-CimMethod GetDeviceProperties`	~2 s
`SELECT <7 columns> … WHERE PNPDeviceID LIKE "PCI\\VEN[_]%"` + the same method	**~1.7 s**

> **THE INTERESTING FAILURE**
>
> The wildcard looked like the fastest option and was in fact doing nothing, because `-KeyName` does not support wildcards. The benchmark was measuring a no-op, and the output still looked plausible. Only checking the *values* caught it — a reminder that a performance test which does not assert on results is measuring the wrong thing.

16.2 Why the per-device path is fast

The per-device property fetch calls the CIM method that the cmdlet wraps: roughly **30 ms per device versus ~1230 ms**. The cmdlet appears to re-resolve the device on every call, while the CIM method takes an already-resolved instance and works from it.

16.3 Where the rest of the time actually went

The second surprise was that the remaining time was not in the per-device calls at all — it was in the `Win32_PnPEntity` enumeration itself. A `WHERE` clause alone barely moves it; *projecting* the seven columns

the module actually reads halves it, because the provider materialises far less per instance.

> **THE WQL ESCAPING IS LOAD-BEARING**
>
> The `LIKE` text has exactly **two backslashes** and `[_]` for the underscore, because `_` is WQL's single-character wildcard. Get the escaping wrong and the query returns **zero rows silently** — which renders as a machine with no PCI bus at all. A test pins the query's row count against a client-side `-like` so that this cannot regress unnoticed.

Parsing `pci.ids` costs about **110 ms**. It was ~50 ms before subsystem names were indexed; the extra 60 ms buys real subsystem names (`Subsystem: Dell PERC H740P [1028:1fd2]`) and was judged worth it. It is not worth optimising further.

16.4 The per-device fetch is parallel

Each `GetDeviceProperties` call costs about **14 ms of fixed round-trip latency plus ~0.9 ms per key**, and the module asks for 34 keys. Serial, that is roughly 1.1 s for 24 devices — and about 12 s for 300.

The fixed part is pure latency, so it overlaps. A **RunspacePool of 8** (Windows PowerShell 5.1 has no `ForEach-Object -Parallel`) took a measured **1.05 s down to 0.47 s** at 24 devices, with identical output.

> **AN OPTIMISATION THAT WAS MEASURED AND REJECTED**
>
> Splitting the fetch in two — core keys now, detail keys only on `-vv` — looked obvious and was worse: the fixed round-trip cost gets paid twice, which made `-vv` *slower* than fetching everything once.

If a machine ever misbehaves under the parallel path, `Get-PciDevice -Serial` or the environment variable `WINLSPCI_SERIAL=1` restores the serial one.

16.5 The fixture replay switch

`WINLSPCI_FIXTURE=<file>` replays a recorded fixture instead of reading the machine. It exists so that the test suite's CLI child processes do not each pay for a live enumeration (§19.2).

> **IT CANNOT PASS FOR REAL OUTPUT**
>
> It is not a user feature and is built so it cannot be mistaken for one: the CLI prints a banner on stderr, and `source` in `-Json` and baseline files becomes `fixture:<name>`.

17 Exit codes and scripting

winlspci is built to be used from scripts as well as by hand, so the exit code carries meaning. There are six of them, and the distinctions between them are deliberate.

CODE	MEANING	WHEN YOU SEE IT
`0`	OK	The command ran and completed successfully
`1`	Matched nothing	A filter — `-s`, `-d`, `-Downtrained`, or `-Attribute` / `-Match` — matched nothing. The command itself worked; the hardware is simply not there
`2`	Impossible or not implemented	The request cannot be answered here (`-x`), or it is a known lspci flag this tool does not implement (`-b`, `-p`, `-P`, … — the full set is in §5.4). An explanation is printed, with a pointer to the nearest equivalent
`3`	Differences found	`-Diff` or `-Watch` found something that changed. `0` means the two sides are identical. See chapter 12
`64`	Usage error	An unknown option, or a selector that is not valid hex
`70`	Enumeration failed	The WMI enumeration itself failed — a Windows/WMI error, *not* an empty machine. Retry, or check the Winmgmt service. Also raised when no remote node could be reached (chapter 13)

17.1 Reading the difference between them

Each code answers a different question, which is the whole point of having six:

1 — about the machine
The command was fine. The hardware you asked about is not present.

2 — about the tool
The command was understood, and this tool will not or cannot answer it.

3 — about the comparison
Everything worked, and the two sides are not the same.

64 — about the command
The command was not understood at all: a typo, or a selector that is not hex.

70 — about Windows
The enumeration itself failed. Nothing can be concluded about the hardware from this — which is exactly why it is not an empty list.

> **THE PAIR THAT MATTERS MOST IN AUTOMATION**
>
> **1 and 70 must never be confused.** Exit 1 says the machine does not have what you asked for. Exit 70 says nobody could find out. A monitoring script that treats an enumeration failure as “card absent” will page someone about hardware that is fine — or, worse, stay silent about hardware that is not.

17.2 Using it in a script

```
lspci -d 11f8: | Out-Null
switch ($LASTEXITCODE) {
    0  { 'switch present' }
    1  { 'switch absent' }
    2  { 'winlspci cannot answer that here' }
    64 { 'bad command line' }
    70 { 'enumeration failed - nothing can be concluded' }
}
```

The *semantics* match Linux, so a colleague's `lspci -d 11f8: || echo "absent"` translates directly — but write the test against `$LASTEXITCODE` as above rather than with `||`, which Windows PowerShell 5.1 does not support.

> **STRUCTURED OUTPUT IN AUTOMATION**
>
> For anything that another program will parse, prefer `-Json` or `-Csv` over `-Delimited`: both quote correctly and neither depends on a fixed column order staying fixed.

18 Troubleshooting

The situations below follow directly from how the tool works. Most of them are the tool being correct rather than the tool being broken.

SYMPTOM	CAUSE AND RESOLUTION
`-s 1` returns something unexpected	Not a defect. Following lspci's grammar, a bare `1` means **device 01 on any bus**, not bus 01. For a bus, write `1:` or `01:` — those two agree. See §15.3.
`-d 8:` does not return every Intel device	Correct. Vendor and device are compared as exact hex IDs, so `-d 8:` is vendor `0008`. Only the *class* field is a prefix match. See §5.1.
`-s01:` or `-s .0` misbehaves, but only when run from a prompt	You are calling `lspci.ps1` directly, so PowerShell's parser sees the value first. Use the `lspci` launcher on PATH, or put a space before the value and quote anything starting with a dot or colon: `-s '.0'`. See §5.2.
A slot has four extra hex digits in front of it, e.g. `7870:00:00.0`	Correct, and expected inside a VM. That prefix is the PCI domain (segment), which Hyper-V and Azure pack into the upper bits of Windows' bus number. See chapter 10.
`-s 00:00.0` returns several devices on a VM	Intended. A selector without a domain matches every domain. Qualify it — `-s 7870:00:00.0` — to pick one. See §10.2.
A device shows no link speed or width	Not a fault. Root ports and chipset devices legitimately expose no link state; the field is *not reported*, which is deliberately distinct from a value of zero. See §15.1.

SYMPTOM	CAUSE AND RESOLUTION
A device reports less than its maximum speed or width	Real, but not automatically a problem — link power management routinely lowers both at idle. Use `-Downtrained` to list every such device, then judge in context.
A device shows *Unknown class* `[????]`	Correct behaviour. Windows reported no class property and no `CC_xxxx` hardware ID for it. Printing class `0000` instead would name a real class the device does not have. See §15.1.
A vendor name looks wrong (e.g. a Microchip part reads PMC-Sierra)	Correct behaviour. `pci.ids` records who registered the vendor ID, not who owns the product line today. See §14.2.
`lspci -x` refuses to run	By design. Configuration-space dumps need a signed kernel-mode driver. Exit code 2. See chapter 2.
A known lspci flag exits 2 with "not implemented"	Intended. Recognised-but-unbuilt flags (`-b` , `-p` , `-P` , `-q` , `-Q` , `-M` , the access-method flags) refuse rather than let the parameter binder guess at a different command. The message names the nearest equivalent. See §5.4.
A command exits 70	The WMI enumeration itself failed — a Windows error, not an empty machine. Retry, or check that the **Winmgmt** service is running. Never treat this as "no devices". See chapter 17.
A `-Diff` or `-Watch` run exits 3	Not an error — it is the answer. Exit 3 means differences were found; `0` means the two sides are identical. See chapter 12.
`-Diff` reports a card as removed *and* added	Devices are matched on `InstanceId` . If the slot and IDs also changed it really is a different device; a reseat that changed only the instance id is reported as `Changed(InstanceId)` . See §12.2.
Every `-Diff` run reports changes on an idle machine	Check whether you passed `-IncludeVolatile` : `PowerState` flips with idle and is ignored by default for exactly that reason. Use `-IgnoreAttribute` for anything else you expect to move. See §12.3.
A remote run prints nothing for one node	That node could not be reached, or its property fetch failed for every device; either way it is warned about and skipped. If *no* node could be reached the command exits 70 rather than printing an empty list. See §13.2.
Filtering with `-s` is no faster than a full listing	Expected. Downstream link attribution needs the whole picture, so the machine is always enumerated in full and the selector applied last. See §9.3.
A banner appears on stderr and `source` reads `fixture:…`	`WINLSPCI_FIXTURE` is set in the environment, so the tool is replaying a recorded capture rather than reading the machine. Clear it. See §16.5.
A command exits 64	Usage error: the option is not recognised at all, or a selector is not valid hex. Distinct from exit 2, which means the option *was* recognised. See chapter 17.
`-D` and `-d` behave differently	They should. Short lspci flags are case-sensitive: `-D` is the domain prefix, `-d` is the ID filter. Long options such as `-Json` are case-insensitive and may be abbreviated. See §5.1.

SYMPTOM	CAUSE AND RESOLUTION
A filter returns nothing and the script treats it as failure	Intended: no match is exit 1, not exit 0. Branch on `$LASTEXITCODE` rather than on empty output. See chapter 17.
`lspci` is not recognised as a command	The `bin` directory is not on PATH for this session. Re-run the PATH line from §3.2, or import the module directly per §3.5.
PATH additions vanished, or the environment broke after setting PATH	Almost certainly `setx`, which truncates PATH at 1024 characters without warning. Use the .NET API shown in §3.2.
Devices show numeric IDs but no names	The `pci.ids` database is missing or damaged. Run `Update-PciIds`, which validates the download before replacing anything and keeps the previous copy as `pci.ids.bak`. Check the bundled date with `lspci -Version`.
A parsed one-liner produces the wrong column	The delimiter is stripped from values, not escaped, so column count is fixed — but an *empty* field is legitimate and means "not reported". If your parser collapses empty fields, use `-Csv` instead. See §8.3.

19 Tests

The test suite runs from a stock Windows install with nothing added.

19.1 Running the suite

```
PS> powershell -ExecutionPolicy Bypass -File tests\Invoke-Tests.ps1
```

The same suite runs on every push under Windows PowerShell 5.1 on a GitHub `windows-latest` runner (`.github/workflows/tests.yml`) — a different machine with a different PCI inventory, which is the point. Chapter 10 exists because that runner found something no physical bench machine would have.

19.2 Recorded fixtures

`tests\fixtures\*.json` are captured PCI enumerations — entities plus their DEVPKEY bags — that the suite replays through `Get-PciDevice` in place of the CIM calls. That way the cases that matter are tested on *every* machine, deterministically:

AZURE'S PACKED BUS NUMBERS · A GERMAN LOCATIONINFO · A PHANTOM DEVICE · THE AUTHOR'S LAPTOP

Each fixture has a `.golden.txt` holding its name-free rendering, so a change that alters output shows up as a golden diff rather than as a silent behaviour change. `tests\Invoke-Tests.ps1 -UpdateGolden` rewrites the goldens *on purpose*, which is the point: updating a golden is a deliberate act with a diff attached to it.

```
PS> tests\Export-PciFixture.ps1 -Path tests\fixtures\<name>.json   # record your own box
```

The replay path is the same `WINLSPCI_FIXTURE` mechanism described in §16.5.

19.3 Why not Pester

No Pester, deliberately. Stock Windows ships Pester 3.4.0, whose `Should Be` syntax is incompatible with Pester 5's `Should -Be`. A Pester suite would therefore pass on the author's machine and fail on a colleague's — precisely the class of misleading result the rest of the tool is built to avoid.

19.4 Hardware-dependent tests

The PCI inventory differs on every machine, so hardware-dependent tests assert on *shape* rather than on values. Where a test genuinely needs a specific device present, it skips with a stated reason rather than failing or, worse, passing vacuously.

> **WHAT THE SUITE PINS**
>
> Among others: that absent values never render as zero (§15.1); that `1:` and `01:` resolve identically (§15.3); that every enumerated device appears in the topology tree exactly once (§9.2); and that vendor `11f8` resolves to any of its three registered names (§14.2).

A Appendix A — Flag quick reference

Two pages to keep next to the bench. Short lspci flags are case-sensitive and combine (`-tv`, `-nnk`, `-vvnn`); long options are case-insensitive and may be abbreviated to a unique prefix.

Selection

`-s <selector>`	Select by slot, using lspci's grammar with every field optional: `01:` (bus), `01:00.0`, `1` (device 01 on any bus), `.0` (every function 0), `:00.0`, `0000:01:00.0`. Padded or unpadded. Without a domain it matches every domain; with one, only that domain
`-d [ven]:[dev][:class]`	Select by ID. Vendor and device are exact hex; class is a prefix
`-Downtrained`	Only devices below their maximum speed or width

*`-s` and `-d` are **display** filters: the whole machine is enumerated first (§9.3), so filtering shortens the answer rather than speeding it up.*

Verbosity and format

`-n / -nn`	IDs only / names and IDs
`-v`	Physical slot, link state (current and max, with power state when it explains a downtrain), device type, driver, status
`-vv`	Adds MPS, MRRS, subsystem, NUMA, capability presence, `ACS:`, serial number, power state, location path
`-vvv`	Everything, plus an explicit list of what is missing
`-k`	Driver and version (the same lines `-v` shows)
`-t`	Topology tree with inline link state and port-type tags
`-D`	Domain on every line (`0000` normally; a real segment on Hyper-V / Azure). Note: `-D` ≠ `-d`
`-m / -mm`	lspci's machine-readable forms, quoted and escaped as lspci does
`-i <file>`	Use an alternate `pci.ids`
`-Json / -Csv`	Structured output; the JSON envelope carries `schemaVersion`, `winlspciVersion`, `generatedAt`, `computerName`
`-Delimited`	`|`-separated records
`-Delimiter / -Header`	Change the separator / name the columns
`-Version`	Tool version and bundled `pci.ids` date

Attribute queries

`-Attribute <names>`	One record per attribute; wildcards accepted
`-Match <regex>`	Filter by value
`-PresentOnly`	Drop attributes the device does not report
`-ListAttributes`	List what is queryable (static list; does not enumerate)

Baseline, diff and watch — chapter 12

`-Baseline <file>`	Write a baseline
`-Diff <file>`	Compare against a baseline; exit 3 if anything changed
`-IncludeVolatile`	Compare `PowerState` too (ignored by default)
`-IgnoreAttribute <names>`	Leave attributes out; wildcards accepted
`-Watch <seconds>`	Re-enumerate on an interval, print only changes
`-Iterations <n>`	Stop after n passes

Remote machines — chapter 13

`-ComputerName <a,b,c>`	Enumerate over WinRM; lines prefixed `host:`
`-Credential <user>`	Prompts; needs a console
`-CimSession <s>`	Cmdlet only: sessions you manage yourself (DCOM works)

Cmdlets

`Get-PciDevice`	Enumerate devices as objects (`-Serial` forces the serial fetch)
`Format-Lspci`	Render device objects in `lspci` style; takes `-Verbosity`
`ConvertTo-PciAttributeRecord`	Expand device objects to one record per attribute
`Export-PciBaseline`	Write a baseline file
`Compare-PciBaseline`	Compare a file against the machine now
`Compare-PciDeviceSet`	Compare any two device sets
`Update-PciIds`	Refresh the bundled `pci.ids` database
`Install-LspciShim`	Put `lspci` on PATH after `Install-Module` (`-Remove` to undo)

Refused / not implemented

`-x, -xxx, -xxxx`	**Refused** — needs a signed kernel-mode driver; exits 2
`-b, -p, -P, -q, -Q, -A, -O, -F, -G, -H1, -H2, -M`	Recognised but not implemented; exits 2 with a pointer to the nearest equivalent

Exit codes

`0`	OK
`1`	A filter matched nothing
`2`	Impossible here, or a known flag not implemented
`3`	`-Diff` / `-Watch` found differences
`64`	Usage error: unknown option, or a selector that is not hex
`70`	The WMI enumeration itself failed — not an empty machine

Environment variables

`WINLSPCI_SERIAL=1`	Force the serial per-device fetch (§16.4)
`WINLSPCI_FIXTURE=<file>`	Replay a recorded fixture; test-suite use only (§16.5)

B Appendix B — Repository layout

Where things live, for anyone reading or extending the source.

```
winlspci.psd1            manifest (exports, version, release notes)
winlspci.psm1            loader: module state, then dot-sources Private\ and Public\
Public\                  exported functions, one file per function or cohesive group
  Get-PciDevice.ps1        enumeration and the device object shape
  Format-Lspci.ps1         lspci-style text
  Format-PciTree.ps1       -t
  Format-PciMachine.ps1    -m / -mm
  Attributes.ps1           ConvertTo-PciAttributeRecord, Get-PciAttributeName
  Format-PciDelimited.ps1  -Delimited
  Compare-PciBaseline.ps1  Export-PciBaseline, Compare-PciBaseline, Compare-PciDeviceSet
  PciIds.ps1               pci.ids parse (vendors, devices, subsystems, classes, prog-ifs),
                             lookups, Update-PciIds
  Install-LspciShim.ps1    puts `lspci` on PATH for Install-Module users
packaging\               release notes: Gallery, scoop manifest, winget, signing
Private\                 internal helpers
  DeviceProperties.ps1     DEVPKEY fetch, BDF, class-from-hardware-id
  Selectors.ps1            -s and -d parsing
  Helpers.ps1              downtrain test, StrictMode-safe field access, text sanitising
bin\lspci.cmd / .ps1     the command-line front end and its argument parser
data\pci.ids             bundled PCI ID database
tests\Invoke-Tests.ps1   the suite (no Pester)
```

Reading it against this manual

`Private\Selectors.ps1`	The `-s` and `-d` grammar described in §5.1 and §15.3
`Private\DeviceProperties.ps1`	The DEVPKEY fetch behind chapter 16's performance work, the BDF assembly behind chapter 10, and the class-from-hardware-ID fallback in §15.1
`Private\Helpers.ps1`	The downtrain test guarded in §15.1, and the text sanitising that strips the delimiter from values (§8.3)
`Public\Format-PciMachine.ps1`	`-m` and `-mm`, lspci's machine-readable forms (§5.1)
`Public\Compare-PciBaseline.ps1`	All of chapter 12 — baseline, diff and watch
`Public\PciIds.ps1`	Everything in chapter 14, including `Update-PciIds`, and the subsystem and prog-if name indexes that cost the extra 60 ms in §16.3
`Public\Install-LspciShim.ps1`	The Gallery shim in §3.4
`packaging\`	Release notes for the Gallery, scoop, winget and signing
`bin\lspci.cmd / .ps1`	The launcher and argument parser discussed in §5.2 — the reason the `.cmd` front end sees your command line before PowerShell's parser does

C Appendix C — Glossary

AER	Advanced Error Reporting. A PCI Express capability for logging and reporting link and transaction errors. winlspci reports whether the capability is *present*; the register contents require a live read and are out of reach.
ACS	Access Control Services. A PCIe capability on a *port* that controls whether peer-to-peer traffic can bypass the IOMMU. Central to DMA isolation and IOMMU grouping. See §11.3.
ARI	Alternative Routing-ID Interpretation. Lets a device expose more than eight functions by reinterpreting the device/function bits.
ATS	Address Translation Services. Lets a device cache IOMMU address translations locally.
AtomicOps	PCIe atomic operations — FetchAdd, Swap, CAS — completed indivisibly at the target.
BDF	`bus:device.function` — the address that identifies a PCI function. Written `01:00.0`, optionally prefixed with a domain: `0000:01:00.0`.
Configuration space	The per-function register space that holds PCI identity and capability structures. Reading it directly from userland is not possible on Windows — the reason winlspci works the way it does.
Domain / segment	The highest-order part of a PCI address. `0000` on an ordinary machine; Hyper-V and Azure report a real segment, packed into the upper bits of Windows' bus number. A non-zero domain is always shown in the slot. See chapter 10.

Downtrained	A link operating below its maximum speed, width, or both. Often power management rather than a fault.
DSN	Device Serial Number — a PCIe capability carrying a 64-bit serial, rendered `00-11-22-33-44-55-66-77`. Present but unpopulated on many devices.
D-state	A PCI power state, D0 (fully on) through D3. winlspci reports the most recent one Windows recorded, not a live read. See §11.5.
GT/s	Gigatransfers per second — the PCIe signalling rate. 8GT/s is Gen3, 16GT/s Gen4, 32GT/s Gen5, 64GT/s Gen6.
Link width	The number of lanes in use, written x1, x4, x8, x16. Reported both as negotiated and as maximum.
MPS	Max Payload Size — the largest data payload the device is configured to move in one transaction.
MRRS	Max Read Request size — the largest read the device will issue in a single request.
InstanceId	Windows' unique identifier for an enumerated device instance. What `-Diff` matches devices on. See §12.2.
MSI / MSI-X	Message Signalled Interrupts. MSI-X supports many more vectors, and winlspci reports the count.
NUMA node	The memory/CPU locality domain a device is attached to. Relevant to bandwidth and latency on multi-socket systems.
pci.ids	The community-maintained database mapping numeric PCI IDs to vendor and device names. Bundled with winlspci; refreshed with `Update-PciIds`.
PnP	Plug and Play — the Windows subsystem that enumerates devices. winlspci reads its data, populated by the PCI bus driver from configuration space at enumeration time.
Prog-if	Programming interface — the third component of the PCI class code, after class and subclass.
Root port	The PCIe port on the host side of a link. Frequently reports no link state of its own, which winlspci renders as *not reported* rather than zero.
SR-IOV	Single Root I/O Virtualization — one physical function presenting multiple virtual functions to guests. winlspci reports `SriovStatus : ok`, or why VFs cannot be enabled.
Subsystem ID	Vendor and device IDs assigned by the board integrator, distinguishing one implementation of a chip from another.

D Appendix D — Licence and credits

Licence

winlspci is released under the **MIT Licence**.

The bundled PCI ID database, `data/pci.ids`, is **BSD / GPL dual-licensed** and comes from the PCI ID Project. It is redistributed under those terms and is not covered by the MIT licence above.

Credits

pci.ids	Maintained by the PCI ID Project — `https://pci-ids.ucw.cz`
lspci	The original Linux utility, part of the pciutils package, whose interface winlspci deliberately follows
Source and issues	`https://github.com/jpmutschler/winlspci`

Scope of this document

> **ABOUT THIS MANUAL**
>
> This manual documents the behaviour of winlspci as released. Chapter 18 and Appendix C organise and expand on behaviour described elsewhere in the documentation set; they introduce no flags, cmdlets, exit codes or output formats beyond those listed in chapters 5, 6, 12, 13 and 17. Where the tool's capabilities are bounded by Windows itself, chapter 2 is the authoritative statement of that boundary.

WINLSPCI — POWERSHELL PCI TOOL (NO DRIVER)

Document revision 1.3 · August 2026 · Specifications subject to change